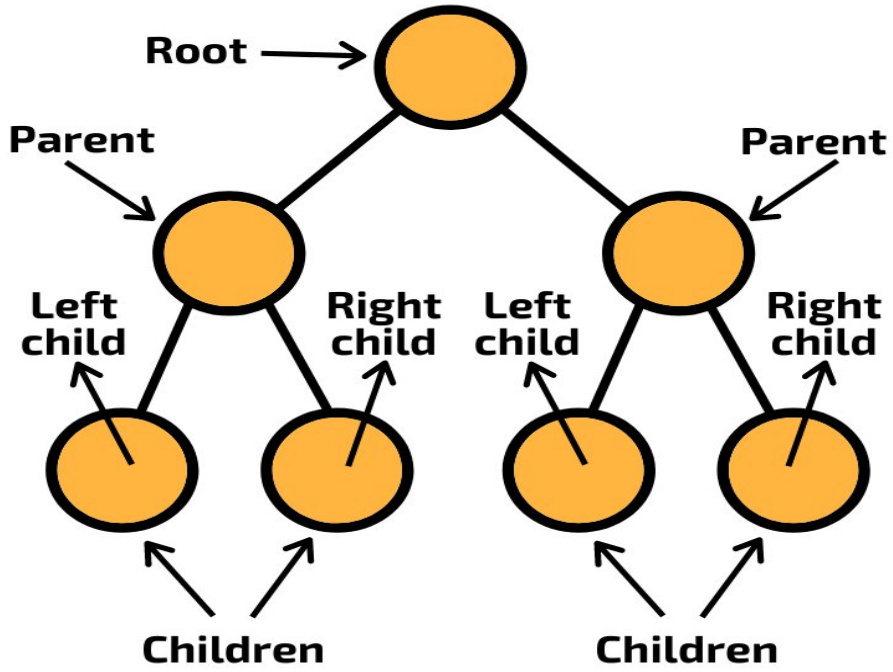
**Trees in Graph Theory**

A **tree** is a special type of graph used to represent **hierarchical structures** such as file systems, organizational charts, and decision trees.

**1 Tree**




**Definition :** A tree is a connected graph with no cycles.

Properties:

- It contains **n vertices and n−1 edges**
- Only **one path exists between any two vertices**

**2 Root of a Tree**

**Definition:** The root is the topmost vertex of a tree from which all other vertices originate.

Example:

If vertex **A** is the starting node, then **A is the root**.

**3 Branch Nodes (Internal Nodes)**

**Definition :** Branch nodes are vertices that have at least one child node.

They are also called **internal nodes**.

Example:

If node **B** has children **D and E**, then **B is a branch node**.

**4 Leaf Nodes (Terminal Nodes)**

**Definition:** A leaf node is a vertex that has no children.

It represents the **end of a branch**.

Example:

Nodes **D, E, F** that have no further nodes.

**Example Tree Structure**

Root:

A

Children:

B, C

Further nodes:

B → D, E
C → F

Classification:

| Node | Type |
|---|---|
| **A** | Root |
| **B, C** | Branch nodes |
| **D, E, F** | Leaf nodes |

**5 Different Representations of a Tree**

Trees can be represented in multiple ways.

**(a) Graphical Representation**

Nodes connected with edges.

Example

```
  /\ \
 D  E  F
```

**(b) Parent Representation**

Each node stores its **parent node**.

Example table:

| Node | Parent |
|---|---|
| **A** | – |
| **B** | A |
| **C** | A |
| **D** | B |
| **E** | B |
| **F** | C |

**(c) Adjacency Matrix Representation**

Vertices are represented using a matrix.

Example:

| | **A** | **B** | **C** | **D** |
|---|---|---|---|---|
| **A** | 0 | 1 | 1 | 0 |
| **B** | 1 | 0 | 0 | 1 |
| **C** | 1 | 0 | 0 | 0 |
| **D** | 0 | 1 | 0 | 0 |

**(d) Adjacency List Representation**

Each vertex stores its connected vertices.

Example:

| Vertex | Adjacent Nodes |
|---|---|
| **A** | B, C |
| **B** | D, E |
| **C** | F |

**Key Properties of Trees**

| Property | Meaning |
|---|---|
| **No cycles** | Tree has no loops |
| **Connected** | Every node reachable |
| **Edges** | n − 1 edges |
| **Unique path** | Only one path between vertices |

---

**6 Eccentricity of a Vertex**

**Definition:** The eccentricity of a vertex in a graph or tree is the **maximum distance from that vertex to any other vertex** in the graph.

**Formula**

$$e(v) = \max\{d(v,u)\}$$

Where
d(v,u) = distance between vertices **v** and **u**.

Meaning: Find the **longest shortest path** from that vertex to any other vertex.

**7 Canter of a Tree**

**Definition:** The center of a tree is the vertex (or vertices) whose eccentricity is minimum among all vertices of the tree.

Meaning:
The node that is **closest to all other nodes in terms of distance**.

Important property:

- A tree has either **one center or two centers**.

**8 Cut Points**

**Definition :** A cut point is a vertex whose removal increases the number of connected components of a graph.

Meaning:
If we remove that vertex, the graph becomes **disconnected**.

Example idea:

A — B — C

If **B is removed**, A and C become disconnected.

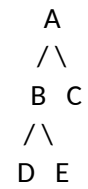
So **B is a cut point**.

**9 Rooted Tree**

**Definition:** A rooted tree is a tree in which one vertex is designated as the **root**, and every edge is directed away from the root.

Meaning: The root acts as the **starting point of the tree hierarchy**.

Example structure

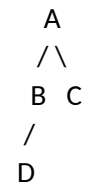



Root = **A**

**1 Level of a Node**

**Definition:** The level of a node is the number of edges in the path from the root to that node.

Example




| Node | Level |
|---|---|
| **A** | 0 |
| **B, C** | 1 |
| **D** | 2 |

**2 Height of a Tree**

**Definition:** The height of a tree is the maximum level of any vertex in the tree.

Meaning: Distance from **root to the deepest leaf node**.

Example

If deepest node level = 3 → height = **3.**

**3 Ordered Tree**

**Definition:** An ordered tree is a rooted tree in which the order of the children of each vertex matters.

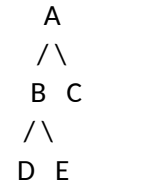
Example: If node A has children **B then C**, this is different from **C then B**.

Order matters.

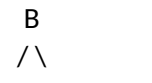
**4 Subtrees**

**Definition:** A subtree is a tree formed by a vertex and all of its descendants in a larger tree.

Example




Subtree rooted at **B**:




**5 m- ary Tree**

**Definition :** An m-ary tree is a rooted tree in which every node has **at most m children**.

Where **m** is a positive integer.

**Example**

Binary tree:

m = 2

Ternary tree:

m = 3

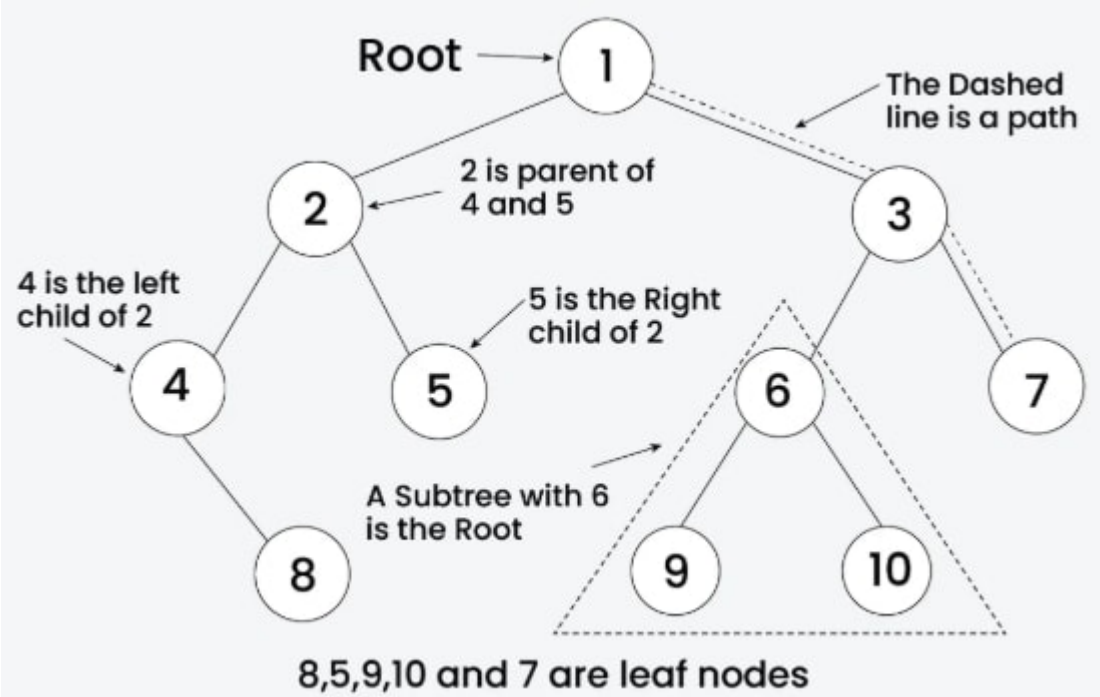
Example structure


Here each node can have **maximum 3 children**.

**Binary Tree**

A **binary tree** is a special type of tree where each node can have **at most two children**.

**6 Binary Tree**




**Definition :** A binary tree is a tree data structure in which each node has at most two children called the left child and the right child.

**7 Structure of Binary Tree**

Each node contains:

- **Root node**
- **Left subtree**
- **Right subtree**

Example structure

```
    A
   / \
  B   C
 / \   \
D   E   F
```

**8 Types of Nodes in Binary Tree**

| **Node Type** | **Meaning** |
|---|---|
| **Root node** | Topmost node |
| **Internal node** | Node with children |

| | |
|---|---|
| **Leaf node** | Node with no children |

Example

| Node | Type |
|---|---|
| **A** | Root |
| **B, C** | Internal |
| **D, E, F** | Leaf |

**9 Important Properties of Binary Trees**

**Property 1 :** Maximum nodes at level **i**

$$2^i$$

**Property 2**

Maximum nodes in binary tree of height **h**

$$2^{h+1} - 1$$

**Property 3**

Maximum number of leaf nodes

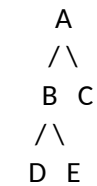
$$2^h$$

**1 Types of Binary Trees**

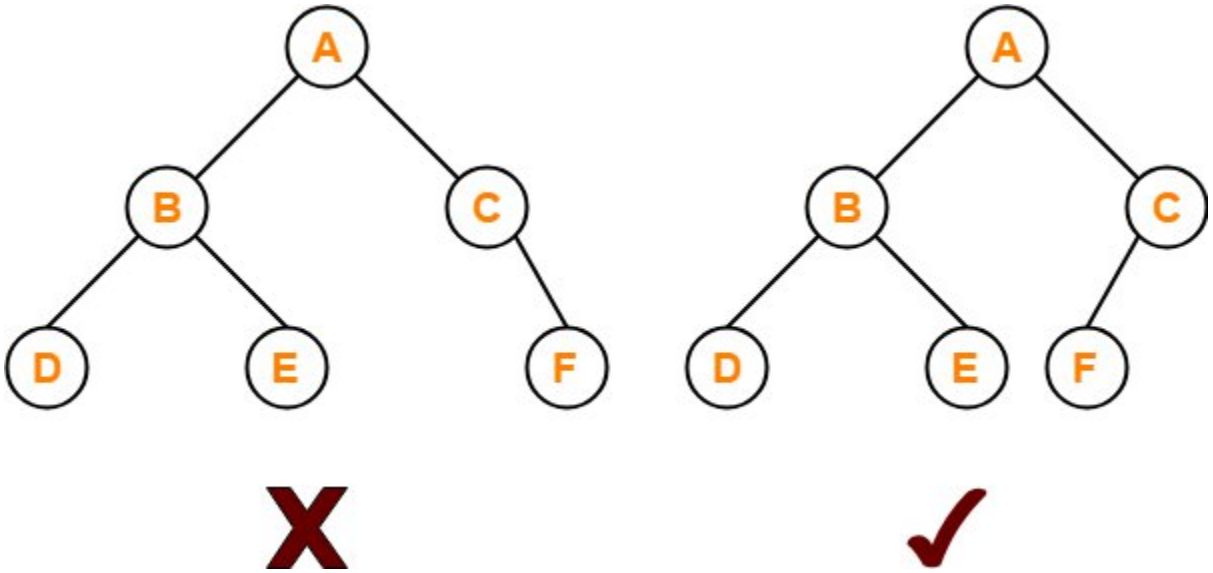
**(a) Full Binary Tree**

**Definition:** A full binary tree is a binary tree where every node has either **0 or 2 children**.

Example:




**(b) Complete Binary Tree**




**Definition:** A complete binary tree is a binary tree where all levels are completely filled except possibly the last level.

The last level is filled **from left to right**.

**(c) Perfect Binary Tree**

**Definition:** A perfect binary tree is a binary tree in which all internal nodes have two children and all leaves are at the same level.

**2 Binary Tree Terminology**

| Term | Meaning |
|---|---|
| **Parent** | Node above |
| **Child** | Node below |
| **Sibling** | Nodes with same parent |
| **Subtree** | Portion of tree |

**3 Applications of Binary Trees**

Binary trees are used in:

- Searching algorithms
- Expression trees
- Binary search trees
- File systems

- AI decision trees

## Converting an m-ary Tree to a Binary Tree

An **m-ary tree** can be converted into a **binary tree** using the **Left-Child Right-Sibling (LCRS)** method.

**4 Idea of Conversion**

In a binary tree, each node has at most **two children**:

- Left child
- Right child

So we represent:

| m-ary Tree Relation | Binary Tree Relation |
|---|---|
| **First child** | Left child |
| **Next sibling** | Right child |

Meaning:

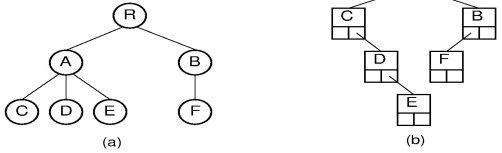
- First child of a node → becomes **left child**
- Next sibling → becomes **right child**

**Example m-ary Tree**

Tree:

```
      A
    / | \
   B  C  D
  / \
 E   F
```

**Converted Binary Tree**







Binary tree representation:

```
          A
```

Explanation

| Node | Binary Tree Relation |
|---|---|
| **B** | Left child of A |
| **C** | Right sibling of B |
| **D** | Right sibling of C |
| **E** | Left child of B |
| **F** | Right sibling of E |

**Steps for Conversion**

1. Connect **each node to its first child** (left link).
2. Connect **each node to its next sibling** (right link).
3. Remove other child connections.

This produces a **binary tree representation of the m-ary tree**.

**Representation of a Binary Tree (Linked List)**

Binary trees are commonly implemented using **linked lists**.

**Structure of Binary Tree Node**

Each node contains:

1. Data
2. Pointer to left child
3. Pointer to right child

**Node Representation**

| Left Pointer | Data | Right Pointer |

**Example Binary Tree**


**Linked List Representation**

| Node | Left Pointer | Data | Right Pointer |
|---|---|---|---|
| **A** | B | A | C |
| **B** | D | B | E |
| **C** | NULL | C | NULL |
| **D** | NULL | D | NULL |
| **E** | NULL | E | NULL |

**Advantages of Linked-List Representation**

- Dynamic memory allocation
- Easy insertion and deletion
- Efficient tree traversal

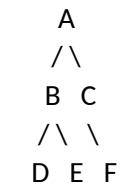
**Tree Traversal Algorithms (Binary Trees)**

Tree traversal means **visiting each node of a tree exactly once in a specific order**.

There are three main traversal methods.

**Example Binary Tree**

Use this tree to understand all traversals.




**1 Preorder Traversal (Root–Left–Right)**

**Definition :** In preorder traversal, the root node is visited first, then the left subtree, and finally the right subtree.

**Order**

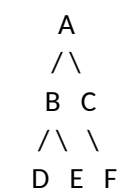
Root → Left → Right

**Steps**

1. Visit root
2. Traverse left subtree
3. Traverse right subtree

**Example**

Tree




Traversal

A → B → D → E → C → F

**2 Inorder Traversal (Left–Root–Right)**

**Definition :** In inorder traversal, the left subtree is visited first, then the root node, and finally the right subtree.

**Order**

Left → Root → Right

**Steps**

1. Traverse left subtree
2. Visit root
3. Traverse right subtree

**Example**

Traversal

D → B → E → A → C → F

**3 Postorder Traversal (Left–Right–Root)**

**Definition:** In postorder traversal, the left subtree is visited first, then the right subtree, and finally the root node.

**Order**

Left → Right → Root

**Steps**

1. Traverse left subtree
2. Traverse right subtree
3. Visit root

**Example**

Traversal

D → E → B → F → C → A

**Comparison Table**

| Traversal | Order | Result for Example Tree |
|---|---|---|
| **Preorder** | Root–Left–Right | A B D E C F |
| **Inorder** | Left–Root–Right | D B E A C F |
| **Postorder** | Left–Right–Root | D E B F C A |

**Applications:**

| Traversal | Use |
|---|---|
| **Preorder** | Copy tree / prefix expression |
| **Inorder** | Binary search tree sorting |
| **Postorder** | Delete tree / postfix expression |

4 Spanning Tree :

**Definition :** A spanning tree of a connected graph is a subgraph that includes all the vertices of the graph and forms a tree.

Meaning:

- Contains **all vertices**
- Has **no cycles**
- Uses **minimum edges to keep graph connected**

**Example**

Original graph:

Vertices
A, B, C, D

Edges
AB, AC, AD, BC, CD

A spanning tree may contain:

AB, AC, AD

This connects all vertices without forming a cycle.

**5 Properties of Spanning Trees**

Important properties frequently asked in exams.

| **Property** | **Explanation** |
|---|---|
| **Contains all vertices** | Every vertex of the graph appears |
| **No cycles** | Spanning tree is acyclic |
| **Minimum edges** | If graph has **n vertices**, spanning tree has **n−1 edges** |
| **Connected** | Every vertex reachable |
| **Removing any edge disconnects tree** | Tree becomes disconnected |
| **Adding any edge creates cycle** | Because tree already minimal |

**Important Theore m**

If a graph has **n vertices**, a spanning tree has:

n−1

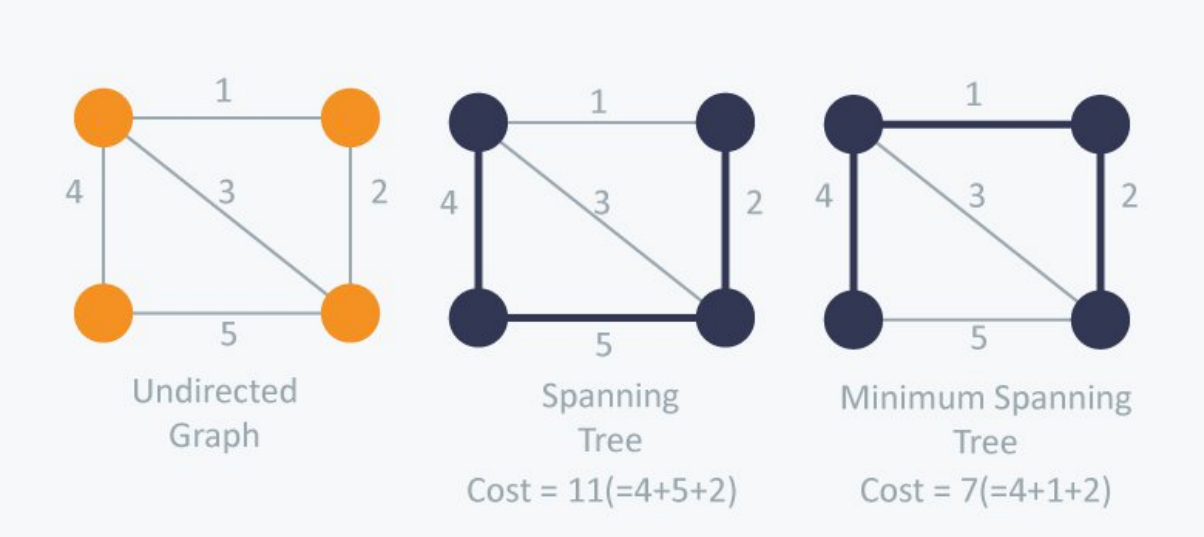
edges.

Example

If vertices = 5

Edges in spanning tree = 4.

**6 Minimum Spanning Tree (MST)**




**Definition :** A minimum spanning tree is a spanning tree of a weighted connected graph in which the sum of edge weights is minimum.

Meaning: Among all possible spanning trees, choose the one with **lowest total weight**.

**Example**

Graph with weights

Edges

AB = 2
AC = 3
BC = 4
BD = 1
CD = 5

Possible spanning tree:

AB + AC + BD

Total weight:

2+3+1=6

If this is the minimum possible → **Minimum Spanning Tree**.

**Applications of Minimum Spanning Tree**

MST is used in:

- Network design (telephone, internet)
- Road and railway planning
- Electrical network design
- Data clustering

- Image segmentation

**Difference**

| Concept | Meaning |
|---|---|
| **Spanning Tree** | Any tree connecting all vertices |
| **Minimum Spanning Tree** | Spanning tree with minimum weight |

**Algorithms to Find MST**

Two famous algorithms:

1. **Kruskal's Algorithm**
2. **Prim's Algorithm**

---

## Kruskal's Algorithm (Minimum Spanning Tree)

**Definition :** Kruskal's Algorithm is a greedy algorithm used to find the **minimum spanning tree (MST)** of a weighted connected graph by selecting edges in increasing order of weight without forming a cycle.

Goal: Construct a **spanning tree with minimum total weight**.

**Steps of Kruskal's Algorithm**

1. List all edges of the graph with their weights.
2. Sort the edges in **ascending order of weights**.
3. Select the smallest edge.
4. Add the edge to the spanning tree **if it does not create a cycle**.
5. Repeat until the spanning tree contains **n − 1 edges**.

Where
n = number of vertices.

**Steps to Write in Exam**

**Step 1**

Write the **adjacency matrix of the graph**.

Let the graph have **n vertices**.

**Step 2**

List all the edges from the adjacency matrix with their weights.

Example:

If matrix entry

$$A_{ij} = w$$

then edge:

$$(v_i, v_j)$$

with weight **w**.

Ignore entries with **0** (no edge).

**Step 3**

Arrange all edges in **ascending order of weights**.

Make a table:

| Edge | Weight |
|---|---|
| **e1** | w1 |
| **e2** | w2 |
| **…** | … |

**Step 4**

Select the **smallest weight edge**.

Add it to the **Minimum Spanning Tree (MST)**.

**Step 5**

Check if adding the edge creates a **cycle**.

- If **cycle forms → reject edge**
- If **no cycle → include edge**

**Step 6**

Repeat the process for the next smallest edge.

**Step 7**

Stop when the number of edges selected becomes:

n - 1

where **n = number of vertices**.

**Example Format to Write in Exam**

**Given Adjacency Matrix**

| | **A** | **B** | **C** | **D** |
|---|---|---|---|---|
| **A** | 0 | 1 | 3 | 0 |
| **B** | 1 | 0 | 2 | 4 |
| **C** | 3 | 2 | 0 | 5 |
| **D** | 0 | 4 | 5 | 0 |

**Step 1 — Extract Edges**

| **Edge** | **Weight** |
|---|---|
| **AB** | 1 |
| **AC** | 3 |
| **BC** | 2 |
| **BD** | 4 |
| **CD** | 5 |

**Step 2 — Sort Edges**

| **Edge** | **Weight** |
|---|---|
| **AB** | 1 |
| **BC** | 2 |
| **AC** | 3 |
| **BD** | 4 |
| **CD** | 5 |

**Step 3 — Select Edges**

| **Edge** | **Action** |
|---|---|
| **AB** | Selected |
| **BC** | Selected |

| | |
|---|---|
| **AC** | Rejected (cycle) |
| **BD** | Selected |

Edges selected = **3 = n−1**

**Minimum Spanning Tree**

Edges: AB, BC, BD

Total weight: 1 + 2 + 4 = 7

**Important Points**

| **Property** | **Explanation** |
|---|---|
| **Algorithm type** | Greedy algorithm |
| **Edge selection** | Smallest weight first |
| **Cycle rule** | Cycles not allowed |
| **Stop condition** | n − 1 edges |

**Time Complexity**

$$O(E \log E)$$

Where
E = number of edges.

**Advantages**

- Simple to implement
- Works well for **sparse graphs**

**Prim's Algorithm (Minimum Spanning Tree)**

**Definition : Prim's Algorithm** is a greedy algorithm used to find the **minimum spanning tree (MST)** of a weighted connected graph by starting from a vertex and repeatedly adding the **minimum weight edge that connects a visited vertex to an unvisited vertex**.

**Goal:** Construct a **spanning tree with minimum total weight**.

**Steps of Prim's Algorithm**

1. Start with any vertex as the **initial vertex**.
2. Mark the starting vertex as **visited**.
3. From all edges connecting visited vertices to unvisited vertices, **choose the edge with minimum weight**.
4. Add that edge and new vertex to the **spanning tree**.
5. Mark the new vertex as **visited**.
6. Repeat the process until **all vertices are included** in the tree.

Where
n = number of vertices.

**Steps to Write in Exam (Using Adjacency Matrix)**

**Step 1**

Write the **adjacency matrix of the graph**.

Let the graph have **n vertices**.

**Step 2**

Choose any vertex as the **starting vertex**.

Mark it as **visited**.

**Step 3**

Look at all edges connecting the **visited vertex to unvisited vertices**.

Select the **minimum weight edge**.

**Step 4**

Add the selected edge to the **Minimum Spanning Tree (MST)**.

Mark the new vertex as **visited**.

**Step 5**

From all visited vertices, check edges connecting to **unvisited vertices**.

Choose the **minimum weight edge**.

**Step 6**

Repeat the process until the number of edges in MST becomes:

n−1

where **n = number of vertices**.

**Example Format to Write in Exam**

**Given Adjacency Matrix**

| | **A** | **B** | **C** | **D** |
|---|---|---|---|---|
| **A** | 0 | 1 | 3 | 0 |
| **B** | 1 | 0 | 2 | 4 |
| **C** | 3 | 2 | 0 | 5 |
| **D** | 0 | 4 | 5 | 0 |

**Step 1 — Start from Vertex A**

Visited = {A}

Possible edges

| **Edge** | **Weight** |
|---|---|
| **AB** | 1 |
| **AC** | 3 |

Choose **AB (1)**.

**Step 2**

Visited = {A, B}

Possible edges

| **Edge** | **Weight** |
|---|---|
| **AC** | 3 |
| **BC** | 2 |
| **BD** | 4 |

Choose **BC (2)**.

**Step 3**

Visited = {A, B, C}

Possible edges

| Edge | Weight |
|---|---|
| **BD** | 4 |
| **CD** | 5 |

Choose **BD (4)**.

**Step 4**

Visited = {A, B, C, D}

Number of edges = **3 = n − 1**

Stop.

**Minimum Spanning Tree**

Edges:

AB, BC, BD

Total weight:

$$1+2+4=7$$

**Important Points**

| Property | Explanation |
|---|---|
| **Algorithm type** | Greedy algorithm |
| **Starting point** | Any vertex |
| **Edge selection** | Minimum edge connecting visited to unvisited |
| **Cycle rule** | Cycles automatically avoided |
| **Stop condition** | n − 1 edges |

**Time Complexity**

$$O\left(V^2\right)$$

Using adjacency matrix.

Where
V = number of vertices.

**Advantages**

- Easy to implement with adjacency matrix
- Efficient for **dense graphs**
- No need to sort all edges (unlike Kruskal)

---

## Key Difference Between Kruskal and Prim

| Feature | Kruskal's Algorithm | Prim's Algorithm |
|---|---|---|
| **Approach** | Edge-based | Vertex-based |
| **Start point** | No starting vertex needed | Must start from a vertex |
| **Edge selection** | Smallest edge in the whole graph | Smallest edge connected to visited vertex |
| **Cycle check** | Must check for cycles | Cycle avoided automatically |
| **Data structure** | Uses edge list | Uses adjacency matrix / priority queue |
| **Best for** | Sparse graphs | Dense graphs |

**One-Line Memory Trick**

**Kruskal:** smallest edge anywhere
**Prim:** grow tree from a vertex

**Reachability (Graph Theory)**

Reachability describes **whether one node in a graph can reach another node through a path**.

It is mainly used in **directed graphs (digraphs)**.

**1 Definition of Reachability**

**Definition :** A vertex v is said to be **reachable** from another vertex u if there exists a path from u to v in the graph.

Meaning:

If we can travel from node **u to v** following edges, then **v is reachable from u**.

**Example**

Graph

A → B → C

Reachability:

- B is reachable from A

- C is reachable from A
- C is reachable from B

But

- A is **not reachable from C**

**2 Reachable Node**

**Definition :** A node v is called a **reachable node** from node u if there exists a path from u to v.

In simple words:

If a node can be **visited starting from another node**, it is a reachable node.

**Example**

Graph

A → B → C → D

Reachable nodes from **A**

B, C, D

So

| Starting Node | Reachable Nodes |
|---|---|
| **A** | B, C, D |

**3 Reachable Set of a Node**

**Definition:** The reachable set of a node is the set of all vertices that can be reached from that node by following paths in the graph.

It contains **all reachable nodes from that vertex**.

**Example**

Graph

A → B → C
↓
D

Reachable set of **A**

R(A)={B,C,D}

Reachable set of **B**

R(B)={C,D}

Reachable set of **C**

R(C) = ∅

---

**1 Geodesic (Shortest Path)**

**Definition :** A **geodesic** between two vertices is the shortest path connecting those vertices in a graph.

Meaning:

Among all possible paths between two nodes, the path with **minimum number of edges** is called the **geodesic**.

**Example**

Graph

A → B → C → D
 \ /
 ----------

Possible paths from **A to D**

1. A → B → C → D (length = 3)
2. A → D (length = 1)

Shortest path = **A → D**

So **A → D is the geodesic**.

**2 Distance Between Two Nodes**

**Definition :** The distance between two vertices u and v is the length of the shortest path (geodesic) between them.

Notation

D ( u, v )

Meaning:

Distance = **number of edges in the shortest path**.

**Example**

Graph

A → B → C → D

Distance values

| Nodes | Distance |
|---|---|
| **d(A,B)** | 1 |
| **d(A,C)** | 2 |
| **d(A,D)** | 3 |

**3 Properties of Reachability**

Reachability has several important properties.

| Property | Explanation |
|---|---|
| **Reflexive** | Every vertex is reachable from itself |
| **Transitive** | If A reaches B and B reaches C, then A reaches C |
| **Connectivity** | Reachability determines connectivity of graph |

**Example of Transitive Property**

Graph

A → B → C

A reaches B
B reaches C

Therefore

A reaches C

**4 Strongly Reachable**

**Definition :** Two vertices u and v are strongly reachable if there exists a path from u to v and also a path from v to u.

Meaning:

Both vertices can **reach each other**.

**Example**

Graph

A → B
↑ ↓
D ← C

A can reach B
B can reach C

C can reach D
can reach A

So all vertices are **strongly reachable**.

**5 Weakly Reachable**

**Definition :** Two vertices are weakly reachable if they become connected when the directions of edges are ignored.

Meaning: Ignore arrow directions → graph becomes connected.

**Example**

Graph

A → B ← C

Ignoring directions

A — B — C

So A and C are **weakly reachable**.

---

**Triangle Inequality (Graph Theory)**

**Definition :** The triangle inequality states that for any three vertices u, v, and w in a graph, the distance from u to v is less than or equal to the sum of the distances from u to w and from w to v.

Mathematically:

$$d(u,v) \leq d(u,w) + d(w,v)$$

Where

- d(u,v) = distance between vertices u and v

**Meaning :** The **shortest path between two vertices is never longer than going through another intermediate vertex**.

In other words:

Direct shortest distance ≤ indirect path distance.

**Example**

Graph

A —— B —— C
\ /
---

Distances:

| Pair | Distance |
|---|---|
| **d(A,B)** | 1 |
| **d(B,C)** | 1 |
| **d(A,C)** | 2 |

Check triangle inequality:

$$d(A,C) \leq d(A,B) + d(B,C)$$

$$2 \leq 1 + 1$$

$$2 \leq 2$$

Condition satisfied.

**Intuition**

Just like a triangle in geometry:

Direct side ≤ sum of other two sides.

Same idea applies to **graph distances**.

**Key Points for Exam**

- Used in **shortest path theory**
- Applies to **distance between vertices**
- Ensures shortest paths behave consistently

---

**Node Base (Graph Theory)**

**Definition :** A **node base** of a directed graph is a set of vertices such that every vertex in the graph is reachable from at least one vertex in that set.

Meaning: If we start from the vertices in the **node base**, we can reach **all other vertices in the graph**.

**Simple Explanation**

Node base = **minimum starting nodes needed to reach the entire graph**.

If you know those starting nodes, you can **visit every vertex**.

**Example**

Graph

A → B → C
D → E

Reachability:

- From **A** → B, C
- From **D** → E

So the **node base** is:

$$\{A, D\}$$

Because starting from A and D we can reach **all nodes**.

**Important Property**

A node base is often formed by selecting **one vertex from each strongly connected component that has no incoming edges from other components**.

**Another Example**

Graph

A → B → C
↑ ↓
D ←------

All nodes can reach each other.

So **node base = {A}** (any one vertex works).

---

**Warshall's Algorithm Flowchart :** Warshall's algorithm computes the **reachability matrix** (transitive closure) from the **adjacency matrix** of a directed graph.

**Explanation of Flowchart Steps**

| Step | Operation |
|---|---|
| **Start** | Begin algorithm |
| **Input** | Read adjacency matrix |
| **Initialize** | W = adjacency matrix |
| **Outer loop** | Select intermediate vertex k |
| **Middle loop** | Iterate through rows (i) |
| **Inner loop** | Iterate through columns (j) |
| **Update rule** | W[i,j] = W[i,j] OR (W[i,k] AND W[k,j]) |
| **End** | Final matrix = reachability matrix |

**Important Formula Used**

$$W_{ij} = W_{ij}\ OR\ \left(W_{ik} \wedge W_{kj}\right)$$

Meaning:

A path exists between **i and j** if:

- direct path exists OR
- path exists through vertex **k**

**Result**

Final matrix produced by the algorithm is called the **Reachability Matrix**.

It shows whether a **path exists between every pair of vertices**.

**Key Exam Points**

- Input → **Adjacency matrix**
- Output → **Reachability matrix**
- Uses **Boolean operations (AND, OR)**
- Works using **three nested loops**